Detecting Machine-Generated Code Across Programming Languages

Melisa Marian and Alexandra Nanu

Babeș-Bolyai University, Faculty of Economics and Business Administration, Cluj-Napoca, Romania

Abstract. The rapid development of Large Language Models for code generation made spotting machine-generated code increasingly harder, which has direct implications on both education, raising concerns about the integrity of student evaluation, and the industry, affecting code quality and vulnerability checks. In this article, we analyze the detection of machine-generated code in multilingual scenarios, using a subset of the SemEval dataset from CoDet-M4, which was conceived to test generalization toward unseen languages. We compare a classic baseline, TF-IDF with logistic regression with pre-trained neural encoders (CodeBERT and GraphCodeBERT). The results show high performance on seen languages, but steep deprecation on unseen languages, confirming the difficulty of cross-language generalization. Structure modeling with GraphCodeBERT results in moderate improvements, while a behavioral analysis suggests a prediction bias associated with the length of code snippets.

Keywords: Machine-Generated Code Detection · Large Language Models · Cross-Language Generalization · CodeBERT · GraphCodeBERT.

1 Introduction

Recent progress in Large Language Models (LLMs) has fundamentally transformed the way in which source code is written, ranging from basic recommendations on how the code ought to be organized, to systems fully capable of generating entire solutions for programming tasks, reaching an almost competitive level. This trend is supported by advances in such systems, now capable of solving complex tasks, such as AlphaCode [11], as well as the development of high performance general models, used on a large scale, such as GPT-4 [14] and its later versions. In this context, the border between human-written and machine-generated code becomes more difficult to trace, while automated generation gains practical and scientific importance.

Our motivation is twofold: in education, the use of generative instruments can affect the integrity of student evaluation, as current detectors can have deprecated performance in real scenarios, especially when code is partially assisted [17]; in the industry, the distinction between human-written and machine-generated code is relevant for quality assessments and vulnerability checks [3].

From a scientific perspective, robust detection is necessary for correct model evaluation, which requires the construction of curated datasets. In this article, we analyze machine-generated code detection in multilingual settings, using a subset of the SemEval dataset, derived from the Code Detection M4 Benchmark (CoDet-M4) [15]. This resource allows for direct evaluation of generalization between seen and unseen programming languages, aligning with recent resources that aim at more realistic evaluation [16].

Our main objective is the comparison under equitable conditions, of a classic baseline, with some pre-trained neural encoders, aiming to answer the following Research Questions (RQ):

- **RQ1:** How effectively can classical and neural models distinguish between human-written and machine-generated code when training and evaluation are conducted on the same programming languages?
- **RQ2:** How does detection performance change when models trained on a subset of programming languages are evaluated on previously unseen languages?
- **RQ3:** Does modeling code structure and data-flow information improve robustness to unseen programming languages?
- **Supplementary Analysis:** How does the code length influence the prediction behavior of the models in unseen language settings?

While the answers to these questions are relevant for researchers developing benchmarks and resources for machine-generated code detection, since we highlight error patterns and conditions in which models base their verdict on fragile clues, our work can also be interesting for code intelligence researchers, as well as code robustness evaluators.

This article is organized as follows: the Related Work section summarizes the relevant literature regarding machine-generated code detection and evaluation of such detectors; in the Data section, we describe the used dataset and a pre-processing and exploratory analysis; the Methodology section presents the models and the experimental protocol, while the Results and Discussions section responds to the research questions by analyzing in-distribution experiments, evaluation on unseen languages, and general behavior.

2 Related Work

This chapter offers a structured analysis of the existing literature on LLM-generated code detection. Unlike classic chronological layouts, the articles are organized on the basis of their methodological approach, in an effort to highlight both convergence and current limitations. In particular, the literature leaves open how well detectors generalize when evaluated on programming languages not seen during training, and it is also unclear whether incorporating code structure reduces reliance on superficial clues, which motivates the research questions covered in this article.

2.1 Machine-Generated Code Detection

The first systematic effort on machine-generated content detection focused on natural language, highlighting statistical differences between the lexical choices humans make, as opposed to those made by machines. In this sense, GLTR [5] shows that generative models favor tokens with higher probability more often than people.

Subsequently, methods such as DetectGPT [12] and Ghostbuster [18] formalized this observation through the analysis of the properties of in-distribution probability. However, recent evaluations such as the M4 benchmark [21] prove that the majority of text detectors suffer significant performance drops in multi-domain and multi-language settings, pointing to generalization issues.

A complementary direction is fine-grained detection that aims to localize or more clearly explain what parts of the text seem machine-generated, offering more than just a final verdict. [1]. Such instruments show that performance and interpretability strongly depend on domain and language, suggesting that the same generalization problem is expected in the case of code generation [21].

Detection of machine-generated code creates a distinct challenge due to the strict syntax constraints and multiple levels of representation. Early approaches focused primarily on traits such as function length, operator, or complexity metrics. Idialu et al. [8] show that such characteristics can differentiate human-written code from GPT-4-generated code in controlled contexts, but do not transfer beyond its training scope. These results suggest that multiple signals exploited by detectors are dependent on the training set distribution and can rapidly degrade when the programming language or the context changes, which exposes a problem of generalization toward unseen languages.

A more recent direction uses supervised neuronal models trained on tokenized code representations. GPTSniffer [13], based on CodeBERT, demonstrates that models pre-trained on code can capture subtle clues of automated generation. However, their evaluation shows a high sensitivity to changes in domain or generator.

Moreover, Pan et al. [17] assess the limitations of existing detectors in real educational contexts, where the code could potentially be partially assisted or even modified by users. Xu et al. [22] propose a contrasted approach for the separation between human-written and machine-generated code representations, resulting in some improvements, but remaining dependent on the distribution of training data. Recent studies such as CoDet-M4 [15] confirm that performance significantly drops in multi-language, multi-generator, and mixed authorship settings, highlighting the need for more robust detection frameworks.

2.2 Models for Code Detection

The performance of modern models for machine-generated code detection is strongly tied to the code representations that are being employed. Unlike natural text, code is characterized by a formal structure and explicit semantic relations, which inspired the development of specialized models for its analysis.

CodeBERT [4] is a representative example of a token-based model pre-trained on natural language and code, proving that statistical and stylistic patterns can be efficiently captured at a sequential level, but it does not explore the explicit internal structure of programs. This limitation is relevant in multilingual scenarios, where syntactic differences can mask semantic similarity.

To integrate semantic information, GraphCodeBERT [7] introduces data flows that model the relations between variables, capturing more profound logical dependencies. A more generalized approach is UniXcoder [6], which proposes a cross-modal representation, combining sequential and structural information in a unified framework. The literature suggests that such models are more robust to language and style variants, as they can potentially yield better generalization toward unseen languages.

2.3 Benchmarks and Evaluation Resources

The evaluation of machine-generated code detection models is difficult due to the diversity of programming languages. Therefore, recent literature proposes benchmarks that go beyond mono-linguistic evaluation. MultiPL-E [2] shows that the performance of the models varies significantly between languages, even for the same task, highlighting the importance of multilingual evaluation. In addition to this, XCodeEval [9] promotes execution-based evaluation, pointing out that some limitations cannot be observed using superficial metrics.

The robustness of the models is analyzed in ReCode [19], showing significant performance drops under slightly modified conditions. Other resources, such as NaturalCodeBench [23] and HumanEval-X [24], show discrepancies between artificial benchmarks and real usage, such as the increased difficulty introduced by multilingual settings. In general, these frameworks provide the necessary context for a critical interpretation of the experimental results.

3 Data

This section describes the used dataset, how it is organized and pre-processed, and the main statistical and structural characteristics relevant for our experimental setup. The dataset is derived from the SemEval shared task on automatically generated code detection, particularly subtask A, and was obtained from the official challenge resources [15]. We employ this dataset because it is standardized, publicly available, and covers multiple programming languages, which makes it suitable for evaluating the ability of models to generalize across languages.

We also perform Exploratory Data Analysis (EDA) to examine the distribution of programming languages, the balance between human-written and automatically generated code, as well as the variation in code snippet length. These properties are essential to understand potential sources of bias and limitations in the dataset. The resulting analysis supports the suitability of data for investigating cross-language generalization in the detection of automatically generated code and motivates the methodological choices presented in the following chapters.

3.1 Dataset Description and Task Framing

The task addressed in this study is formulated as a binary classification problem: given a code fragment, the goal is to predict whether it was written by a human or generated automatically by an LLM. Each sample in the dataset consists of a source code fragment and an associated binary label that indicates its origin.

The dataset is derived from the corpus introduced in [15], and was specifically designed to evaluate the ability of models to generalize across different programming languages and environments. The code snippets come mainly from algorithmic problem-solving tasks, which ensures a relatively uniform semantic context, while still allowing substantial variation in coding style and implementation details.

The data is split into three subsets: a training set, a validation set, and a test set. A key aspect of the experimental setup is the partition of programming languages into "seen" and "unseen" languages. The languages in the training and validation sets (seen languages) are Python, Java, and C++, while the test set contains additional languages that do not appear during training, namely Go, PHP, C #, C and JavaScript. The division allows for the explicit evaluation of cross-language generalization, the central objective of our study. Table 1 summarizes the number of samples in each subset, broken down by label (human vs automatically generated) and by programming language.

Table 1. Distribution of dataset splits by programming language and class label

Split	Language	n	Human (n)	Human (%)	Machine (n)	Machine (%)
Train	Python	457,306	218,103	47.69	239,203	52.31
	Java	19,302	9,225	47.79	10,077	52.21
	C++	23,392	11,147	47.65	12,245	52.35
Validation	Python	91,461	43,620	47.69	47,841	52.31
	Java	3,860	1,845	47.80	2,015	52.20
	C++	4,679	2,230	47.66	2,449	52.34
Test	Python	303	223	73.60	80	26.40
	Java	256	215	83.98	41	16.02
	C++	75	62	82.67	13	17.33
	C	51	41	80.39	10	19.61
	C#	122	94	77.05	28	22.95
	Go	60	44	73.33	16	26.67
	JavaScript	85	53	62.35	32	37.65
	PHP	48	45	93.75	3	6.25

In addition to the labeled subsets, we employ an extra unlabeled dataset provided by the challenge, which contains a larger volume of code fragments. This data is not used for training or model selection; instead, it serves exclusively for additional behavioral analysis (blind tests), aimed at investigating the stability of model predictions, taking into consideration factors such as code length and language. Similarly to other recent benchmarks for detecting AI-generated

code, which cover broader sets of generators, domains, and adversarial or hybrid scenarios [16], our setup focuses on a controlled SemEval subset designed specifically to stress cross-language generalization.

3.2 Pre-processing and Cleaning

The data were subjected to a standardized cleaning and filtering process to remove non-informative samples and ensure experimental consistency. All code snippets were converted to text (string) format, while missing or invalid samples were removed.

An important step in pre-processing is filtering based on code fragment length. Very short fragments, which do not contain enough semantic context for an informed decision, were removed, as well as extremely long fragments, which can introduce noise or excessive computational constraints. The minimum and maximum length thresholds were empirically established and applied uniformly to all subsets. The percentage of samples removed by this filter is low ($<2\%$), indicating that most of the data is valid and informative, and is comparable to the fraction of examples filtered out by length and structural constraints in previous work [15] [16].

The distribution of programming languages in the training set is highly unbalanced, with Python being predominant compared to Java and C++. To reduce the risk of models learning trivial correlations based on language frequency and to facilitate a more stable training process, we applied a sub-sampling strategy to the Python fragments in the training set. Thus, the number of Python samples was reduced to a level comparable to that of the other seen languages, while maintaining the original proportion between the "human" and "automatically generated" classes Table 2. This adjustment does not affect the validation and test sets and is intended solely to improve linguistic balance during training.

For Transformer models, additional tokenization-related constraints were applied. The code snippets were processed with each model's dedicated tokenizer, using a fixed maximum sequence length. Snippets exceeding this limit are truncated and shorter ones are padded, using the same policy for all experiments. This choice ensures comparability between models and control over computational cost.

To prevent data leakage, we kept the original train, validation, and test splits provided by the challenge unchanged and did not use test data at any training or hyperparameter selection stage [15]. All experiments were carried out using a fixed seed for random number generators, ensuring the reproducibility of the results, according to recent large-scale detection studies [16].

3.3 Exploratory Data Analysis (EDA)

To better understand the structure of the dataset and potential sources of bias, we performed an EDA focused on the distribution of programming languages, class balance, and length of code fragments. The analysis provides a quantitative

Table 2. Training set statistics after language-balancing sub-sampling

Language	n	Human (n)	Human (%)	Machine (n)	Machine (%)
Python	50,000	23,847	47.69	26,153	52.31
Java	19,302	9,225	47.79	10,077	52.21
C++	23,392	11,147	47.65	12,245	52.35
Total	**92,694**	**44,219**	**47.70**	**48,475**	**52.30**

basis for the pre-processing decisions described previously and for the evaluation protocol adopted in later chapters.

The language distribution analysis shows a clear predominance of Python in the original training set, followed by Java and C++. This pattern is consistent with the prevalence of Python in many contemporary programming datasets and educational settings, but also introduces a potential bias toward the dominant language. As summarized in Table 1, unseen languages (Go, PHP, C#, C, and JavaScript) appear exclusively in the test set, making their evaluation significantly more challenging and directly relevant for the study of cross-language generalization. The initial skew towards Python in the training data motivates the sub-sampling strategy previously applied. The resulting balanced training distribution is reported in Table 2.

In terms of labels, the dataset is relatively balanced, with similar proportions of human-written and automatically generated code in all splits 1. This reduces the risk of models achieving high performance through trivial strategies based solely on class frequency, aligning with the design of prior AI-generated code detection datasets in educational and competitive contexts [8], [17].

Regarding code fragment length, statistical analysis reveals substantial variability, with both very short and very long snippets present in all subsets. This is important because input length can strongly influence model behavior, particularly in generalization scenarios. The length distributions differ across languages, as well as between seen and unseen settings, suggesting that models may face different difficulty levels depending on the typical size of code in each language. For this reason, fragment length is subsequently used as an additional axis of analysis when evaluating model behavior in unseen languages, complementing the standard aggregate metrics reported in later sections.

4 Methodology

This section describes the experimental framework used for machine-generated code detection, including the analyzed models, the training and validation protocol, and the metrics evaluated. The methodological design is chosen in a way that allows for a controlled evaluation of differences between classic and neural models (RQ1), the capacity to generalize to unseen languages (RQ2), and the performance changes when using structured information (RQ3).

4.1 Classical Baselines

To offer an interpretable reference point, we have included classic models, widely used in early literature on machine-generated code detection, which rely on the assumption that generative models produce outputs with certain stylistic and structural patterns, in a way that differs from how humans would do it.

We choose Term Frequency-Inverse Document Frequency with Logistic Regression (TF-IDF + LR) as a lightweight and widely used linear baseline for code and text classification, providing a transparent reference for RQ1 [10]. This baseline is also computationally inexpensive, making it suitable as a reproducible reference point against heavier neural encoders. As a result, we represent each code snippet as a bag-of-tokens using TF-IDF weighting and we train a Logistic Regression classifier, capturing superficial lexical and formatting cues (token frequency patterns) without modeling code semantics or structure.

Such approaches have been successfully used in controlled studies, demonstrating the ability to distinguish human-written code from GPT-4-generated code, but with significant limitations outside the scope of training [8]. Evaluations in real-world situations, such as in an educational environment, point toward an accentuated performance loss when the code is partially assisted or modified [17]. In this article, these models are used as baselines, making them useful for comparison with existing work.

4.2 Neural Code Encoders

When it comes to neural approaches, we have selected models pre-trained on source code, important for two distinct paradigms of representation:

- **token-based models**: treating code as a token sequence;
- **structure-aware models**: explicitly integrating structural information.

The comparison of models exclusively based on token sequences and those that integrate structural information is used to test if the structured representation leads toward better robustness in scenarios focusing on unseen languages.

CodeBERT [4] is used as a canonical example of token-based encoders, being pre-trained on both natural language and code snippets, demonstrating a solid performance for classification and code analysis tasks. This captures statistical patterns, without explicitly modeling the internal semantic relations of the programs.

In contrast, GraphCodeBERT introduces data flow information, using dependencies between variables and operations, which allow for a richer semantic representation of the code [7].

CodeBERT and GraphCodeBERT form a controlled comparison: both are pre-trained Transformer encoders for code, but GraphCodeBERT additionally incorporates data-flow signals. This pairing allows us to isolate the impact of structural information on cross-language robustness (RQ3) while keeping the overall modeling framework comparable.

In order to ensure an equitable evaluation, neural models are used as encoders followed by a simple classifier, avoiding the addition of other components that could introduce artificial advantages in some cases.

4.3 Training and Validation Protocol

The experimental protocol is conceived to evaluate the generalization capacity of the models, and not just their performance under ideal circumstances. The validation strategy follows solid practices of such systems, specifically:

- the data is split between training, validation, and testing sets;
- the selection of hyperparameters is exclusively done on the validation set;
- the test set is used a single time for the final report.

Moreover, we respect recommendations related to generated code detection systems, including keeping the same training protocols for all analyzed architectures [20].

The training hyperparameters are selected in such a way as to reflect standard practices in the existing literature, ensuring comparable conditions between the analyzed models. For neural models, we have used a learning rate selected from a learning rate sweep on the interval 1e-5, 2e-5, 3e-5, frequently reported in the literature using CodeBERT and derived models for classification and generated code detection tasks [4], [13].

The number of epochs was fixed at 5, aligning with recent evaluations showing that prolonged training does not bring proportionate improvements and can facilitate overfitting in cross-domain settings [21]. The batch dimension was set to 16 for training and 32 for evaluation, a configuration used in articles that aim to compromise between optimization stability and memory limitations [20].

4.4 Metrics and Statistical Reporting

Performance evaluation is done using standard metrics for binary classification that allow for clear and comparable interpretation of the results: *accuracy*, *precision*, *recall* and *F1-score.*

Taking into account the potential imbalances between classes, F1 is used as the main comparison metric. To reduce the influence of stochastic variations, we fix random seeds and report the results of the best checkpoint selected in the validation set, according to the examples in the literature covering the evaluation of similar neural models [19]. This reporting strategy allows for a rigorous analysis of the differences between model families without reaching premature conclusions.

5 Results and Discussion

This section presents and discusses the obtained experimental results for machine-generated code detection. The analysis is structured around the research questions defined in the introduction, covering the in-distribution performance generalization across different languages, and the impact of structural code modeling.

An additional behavioral analysis examines the influence of code length on the model's prediction capabilities. The results covered in this section are interpreted with respect to the data characteristics and methodological choices previously described.

5.1 RQ1: Performance In-Distribution on Seen Languages

We start by evaluating the capacity of classic and neural models to distinguish between code written by humans and machine-generated code, when both the training and evaluation are done using the same programming languages. This experiment serves as a solid consistency check and establishes a superior performance limit under favorable conditions.

Table 3. In-distribution performance on seen languages (Baseline vs. CodeBERT).

Metric	**TF-IDF + LR**	**CodeBERT**
Accuracy	0.9335	0.9918
Precision (macro avg)	0.9332	0.9916
Recall (macro avg)	0.9339	0.9918
Macro-F1	0.9334	0.9917

Table 3 reports Macro-F1 scores obtained on the *val_seen* set for the baseline model (TF-IDF + LR), reaching high performance, confirming prior suspicions that surface-level stylistic and statistical traits capture relevant clues for machine-generated code detection in controlled environments. However, fine-tuned CodeBERT significantly outperforms the classic baseline, reaching almost perfect Macro-F1 scores.

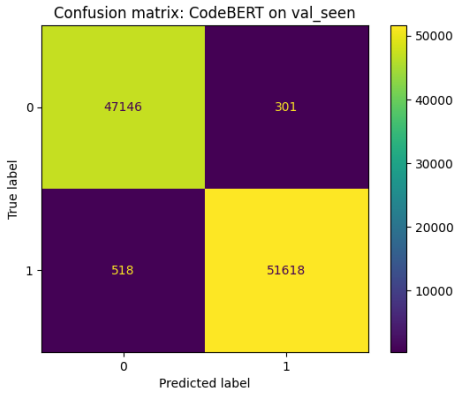

Fig. 1. Confusion matrix of CodeBERT predictions on seen values.

A more detailed perspective on CodeBERT prediction is offered by the confusion matrix in Figure 1, where we observe that both false positives and false negatives are rare, indicating that the model efficiently separates the two classes without counting on trivial decision rules.

Table 4. CodeBERT learning-rate sweep results on the in-distribution validation set.

Learning rate	Best F1	Eval F1	Best checkpoint
1×10^{-5}	0.9909	0.9909	codebert_lr_1e5/23000
2×10^{-5}	0.9913	0.9913	codebert_lr_2e5/23000
$\mathbf{3 \times 10^{-5}}$	**0.9918**	**0.9918**	**codebert_lr_3e5/23000**

The sensitivity to hyperparameters is analyzed with the help of a sweep of the learning rate, summarized in Table 4, where we can also observe the dynamic of the training phase. Although all learning rates lead to high performance in the validation set, the learning rate of 3×10^{-5} constantly obtains the best Macro-F1 score and is therefore selected for the upcoming experiments. The small differences between learning rates suggest that in-distribution performance is not highly sensitive to moderate learning rate changes in our setting.

Overall, these results respond to our first research question affirmatively: both classic and neural models can efficiently detect machine-generated code when the evaluation is done in seen languages, with the neural models displaying a clear performance advantage.

5.2 RQ2: Generalization Across Unseen Languages

Next, we are going to evaluate how the detection performance behaves when models are trained on one subset of the programming languages and are evaluated another one containing exclusively unseen languages. This experiment directly tackles the central problem of generalization, as has been identified in the literature.

Table 5. CodeBERT cross-language performance (Macro-F1) on unseen programming languages.

Language	n	Macro-F1
C	51	0.2917
C#	121	0.2058
Go	60	0.1892
JavaScript	85	0.2308
PHP	48	0.0817

Table 5 shows Macro-F1 scores on the *test_unseen* set for the baseline model, CodeBERT and GraphCodeBERT, both aggregated and split between programming languages (C, C#, Go, JavaScript, PHP). From Figure 2 we can easily

observe a substantial performance drop across all models, compared to the result for seen languages, confirming that generalization between languages represents a significant challenge.

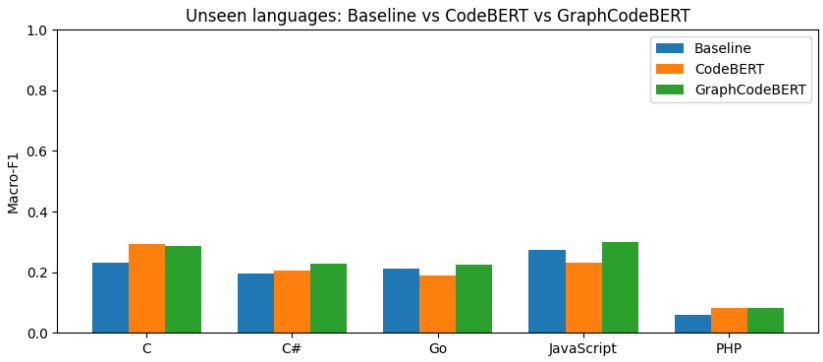

Fig. 2. Macro-F1 performance on unseen programming languages for the baseline, CodeBERT, and GraphCodeBERT models.

The TF-IDF baseline displays a steep deprecation, suggesting that the learned surface-level clues for seen languages do not transfer to new syntactic and stylistic regions. CodeBERT manifests a more robust behavior, obtaining slightly higher Macro-F1 results for the majority of unseen languages, although its performance remains well below the distribution level. These conclusions align with previous evaluation of multilingual robustness in the detection of machine-generated code.

The specific analysis highlights a significant variability: JavaScript and C present relatively better scores, while PHP remains incredibly difficult to predict, regardless of the model. This disparity most likely comes from big differences in syntactic regularity, idiomatic use, and the dimension of the dataset, as well as mismatches between pre-training corpus and target languages.

These results highlight that even if neural models partially alleviate the loss in generalization, detection in unseen languages remains an open problem. Therefore, RQ2 is approached by demonstrating that the detection performance drops considerably when it comes to unseen languages, even when using strong neural encoders.

5.3 RQ3: The Impact of Structural Code Modeling

In order to investigate whether explicit code modeling improves robustness in the face of unseen languages, we compare CodeBERT with GraphCodeBERT, which integrates information of the type of data-flow.

The training dynamic and the selection of hyperparameters for GraphCodeBERT are summarized in Table 6. Similarly to CodeBERT, GraphCodeBERT presents a stable convergence, the learning rate of 2×10^{-5} obtaining the best

Table 6. GraphCodeBERT learning-rate sweep and validation performance on seen languages.

Learning rate	Best val Macro-F1	Epochs	Best checkpoint
1×10^{-4}	0.9367	2	gcb_lr_1e4/checkpoint-5750
3×10^{-5}	0.9917	3	gcb_lr_3e5/checkpoint-11500
$\mathbf{2 \times 10^{-5}}$	**0.9921**	5	**gcb_lr_2e5/checkpoint-23000**

Macro-F1 on the validation set. By consulting Figure 2 and Table 7 we can compare Macro-F1 scores on unseen languages for all three models. GraphCodeBERT consistently over-performs CodeBERT for a majority of languages, with visible improvements especially in the case of JavaScript and C. This result suggests that the integration of structural information and data-flow-type relations help in the abstraction process of the model enough to catch more subtle differences.

Table 7. Macro-F1 performance on unseen programming languages for all evaluated models.

Language	n	Baseline	CodeBERT	GraphCodeBERT
C	51	0.2306	0.2917	0.2873
C#	121	0.1942	0.2058	0.2283
Go	60	0.2105	0.1892	0.2239
JavaScript	85	0.2735	0.2308	0.2983
PHP	48	0.0588	0.0817	0.0817

However, the improvements are not uniform. For PHP, performance stays low, suggesting that structural modeling is not enough to overcome extreme changes in distribution or limited coverage of training data. These conclusions show that even if the representations that are sensitive to structure improve robustness, they do not completely resolve the issue of generalization between languages.

Overall, the results offer a positive result, although nuanced, for the third research question: modeling code structure does improve robustness in relation to unseen languages, but the gains are moderate and language-dependent.

5.4 Behavior of Predictions Based on Code Length

In addition to aggregated performance metrics, we examine the dependence of model predictions on code length by employing a large, blind, unlabeled test set that is stratified according to sequence length.

Figure 3 presents the estimated probability that a fragment can be classified as machine-generated, based on its length. All three models manifest a clear bias for: larger code fragments are more likely to be classified as machine-generated. This tendency is more pronounced for the baseline mode, suggesting a strong

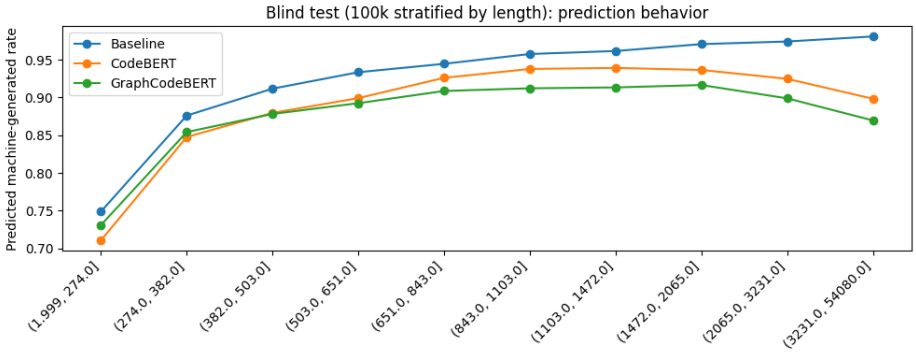

Fig. 3. Prediction behavior as a function of code length on a large unlabeled blind test set (100k samples).

dependence on traits correlated with length. CodeBERT reduces, but does not eliminate, this bias, while GraphCodeBERT presents the most stable behavior on the analyzed intervals. These observations suggest that structural modeling partially decouples prediction from superficial clues related to length.

Although this experiment does not use ground truth labeling, it offers valuable information about the model's behavior, as well as possible weak points related to systemic bias invisible in standard evaluation of labeled data.

5.5 Discussion and Limitations

All in all, the experiments show that neural encoders significantly over-perform classic baselines in distribution scenarios, as well as generalization between languages. Structural modeling improves robustness, supporting the claim that richer representations benefit generalization. However, the performance in unseen languages remains far from satisfactory, highlighting fundamental limitations of current approaches.

Probably a multitude of factors contribute to this limitation. First, the coverage of training data is restricted to a small number of programming languages, which limits the diversity of syntactic patterns from which models can learn. Second, computational constraints have limited the exploration of larger models, longer training sets, and more extended hyperparameter tuning.

All experiments were conducted using the Google Colab environment, which imposed limitations on batch size, sequence length, and training time. In the end, our analysis is limited to binary classification and does not explicitly model hybrid code or partially assisted code, which is the most widespread in today's day and age.

Despite these limitations, the experimental analysis directly responds to the research questions and offers clear proof of the strengths and weaknesses of current AI-generated code detection models. The results support the previously

stated need for more complex multi-lingual training sets and more robust representations to obtain more capable versions of current systems.

6 Conclusion

In this article, we investigate machine-generated code detection in multilingual settings, with a focus on in-distribution performance, generalization toward unseen languages, and the impact of structural code representation. The results clearly show that, under favorable conditions, both classic models, as well as neural ones, can efficiently distinguish between human-written and machine-written code, the encoders offering a clear performance advantage. In generalized scenarios, the performance of all models sharply drops, confirming that generated code detection remains a difficult problem outside the training scope. Comparison of neural models shows that the integration of structured information and data-flow type relations results in consistent improvements, but still far from ideal results.

The complementary behavioral analysis suggests that the predictions of the tested models are influenced by the length of the code snippets, highlighting their bias, which is not captured in standard evaluations on labeled data. Overall, the results directly respond to the formulated research questions, indicating that although neural and structured representation models bring clear benefits, the robust detection of machine-generated code in multilingual settings remains limited by training data coverage and experimental constraints. These conclusions support the need for larger and more diverse datasets, as well as for more rigorous evaluation methods without overpassing the proof produced by presenting our experimental analysis.

References

1. Abassy, M., Elozeiri, K., Aziz, A., Ta, M.N., Tomar, R.V., Adhikari, B., Ahmed, S.E.D., Wang, Y., Mohammed Afzal, O., Xie, Z., Mansurov, J., Artemova, E., Mikhailov, V., Xing, R., Geng, J., Iqbal, H., Mujahid, Z.M., Mahmoud, T., Tsvigun, A., Aji, A.F., Shelmanov, A., Habash, N., Gurevych, I., Nakov, P.: LLM-DetectAIve: a Tool for Fine-Grained Machine-Generated Text Detection. In: Hernandez Farias, D.I., Hope, T., Li, M. (eds.) Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. pp. 336–343. Association for Computational Linguistics, Miami, Florida, USA (Nov 2024). https://doi.org/10.18653/v1/2024.emnlp-demo.35, https://aclanthology.org/2024.emnlp-demo.35/
2. Cassano, F., Gouwar, J., Nguyen, D., Nguyen, S., Phipps-Costin, L., Pinckney, D., Yee, M.H., Zi, Y., Anderson, C.J., Feldman, M.Q., Guha, A., Greenberg, M., Jangda, A.: MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. IEEE Transactions on Software Engineering **49**(7), 3675–3691 (Jul 2023). https://doi.org/10.1109/TSE.2023.3267446, https://ieeexplore.ieee.org/document/10103177/

3. Cotroneo, D., Improta, C., Liguori, P.: Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity. In: 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE). pp. 252–263. IEEE, Sao Paulo, Brazil (Oct 2025). https://doi.org/10.1109/ISSRE66568.2025.00035, https://ieeexplore.ieee.org/document/11229706/?denied=, iSSN: 2332-6549
4. Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M.: CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In: Cohn, T., He, Y., Liu, Y. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2020. pp. 1536–1547. Association for Computational Linguistics, Online (Nov 2020). https://doi.org/10.18653/v1/2020.findings-emnlp.139, https://aclanthology.org/2020.findings-emnlp.139/
5. Gehrmann, S., Strobelt, H., Rush, A.: GLTR: Statistical Detection and Visualization of Generated Text. In: Costa-jussà, M.R., Alfonseca, E. (eds.) Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. pp. 111–116. Association for Computational Linguistics, Florence, Italy (Jul 2019). https://doi.org/10.18653/v1/P19-3019, https://aclanthology.org/P19-3019/
6. Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J.: UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In: Muresan, S., Nakov, P., Villavicencio, A. (eds.) Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 7212–7225. Association for Computational Linguistics, Dublin, Ireland (May 2022). https://doi.org/10.18653/v1/2022.acl-long.499, https://aclanthology.org/2022.acl-long.499/
7. Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S.K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., Zhou, M.: GraphCodeBERT: Pre-training Code Representations with Data Flow (Sep 2021). https://doi.org/10.48550/arXiv.2009.08366, http://arxiv.org/abs/2009.08366, arXiv:2009.08366
8. Idialu, O.J., Mathews, N.S., Maipradit, R., Atlee, J.M., Nagappan, M.: Whodunit: Classifying Code as Human Authored or GPT-4 Generated - A case study on CodeChef problems. In: Proceedings of the 21st International Conference on Mining Software Repositories. pp. 394–406. ACM, Lisbon Portugal (Apr 2024). https://doi.org/10.1145/3643991.3644926, https://dl.acm.org/doi/10.1145/3643991.3644926
9. Khan, M.A.M., Bari, M.S., Do, X.L., Wang, W., Parvez, M.R., Joty, S.: XCodeEval: An Execution-based Large Scale Multilingual Multitask Benchmark for Code Understanding, Generation, Translation and Retrieval. In: Ku, L.W., Martins, A., Srikumar, V. (eds.) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 6766–6805. Association for Computational Linguistics, Bangkok, Thailand (Aug 2024). https://doi.org/10.18653/v1/2024.acl-long.367, https://aclanthology.org/2024.acl-long.367/
10. Kowsari, K., Jafari Meimandi, K., Heidarysafa, M., Mendu, S., Barnes, L., Brown, D.: Text Classification Algorithms: A Survey. Information **10**(4) (2019), https://www.mdpi.com/2078-2489/10/4/150
11. Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., Hubert, T., Choy, P., De Masson d'Autume, C., Babuschkin, I., Chen, X., Huang, P.S., Welbl, J., Gowal, S., Cherepanov, A., Molloy, J., Mankowitz, D.J., Sutherland Robson, E., Kohli, P., De Freitas, N., Kavukcuoglu, K., Vinyals, O.: Competition-level code generation

with AlphaCode. Science **378**(6624), 1092–1097 (Dec 2022). https://doi.org/10.1126/science.abq1158, https://www.science.org/doi/10.1126/science.abq1158

12. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C.D., Finn, C.: DetectGPT: zero-shot machine-generated text detection using probability curvature. In: Proceedings of the 40th International Conference on Machine Learning. pp. 24950–24962. ICML'23, JMLR.org, Honolulu, HI, USA (2023)
13. Nguyen, P.T., Di Rocco, J., Di Sipio, C., Rubei, R., Di Ruscio, D., Di Penta, M.: GPTSniffer: A CodeBERT-based classifier to detect source code written by ChatGPT. Journal of Systems and Software **214**, 112059 (Aug 2024). https://doi.org/10.1016/j.jss.2024.112059, https://linkinghub.elsevier.com/retrieve/pii/S0164121224001043
14. OpenAI, Achiam, J., others: GPT-4 Technical Report. arXiv preprint arXiv:2303.08774 (2024). https://doi.org/10.48550/arXiv.2303.08774, http://arxiv.org/abs/2303.08774
15. Orel, D., Azizov, D., Nakov, P.: CoDet-M4: Detecting Machine-Generated Code in Multi-Lingual, Multi-Generator and Multi-Domain Settings. In: Che, W., Nabende, J., Shutova, E., Pilehvar, M.T. (eds.) Findings of the Association for Computational Linguistics: ACL 2025. pp. 10570–10593. Association for Computational Linguistics, Vienna, Austria (Jul 2025). https://doi.org/10.18653/v1/2025.findings-acl.550, https://aclanthology.org/2025.findings-acl.550/
16. Orel, D., Paul, I., Gurevych, I., Nakov, P.: Droid: A Resource Suite for AI-Generated Code Detection. In: Christodoulopoulos, C., Chakraborty, T., Rose, C., Peng, V. (eds.) Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. pp. 31263–31289. Association for Computational Linguistics, Suzhou, China (Nov 2025). https://doi.org/10.18653/v1/2025.emnlp-main.1593, https://aclanthology.org/2025.emnlp-main.1593/
17. Pan, W.H., Chok, M.J., Wong, J.L.S., Shin, Y.X., Poon, Y.S., Yang, Z., Chong, C.Y., Lo, D., Lim, M.K.: Assessing AI Detectors in Identifying AI-Generated Code: Implications for Education. In: Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training. pp. 1–11. ACM, Lisbon Portugal (Apr 2024). https://doi.org/10.1145/3639474.3640068, https://dl.acm.org/doi/10.1145/3639474.3640068
18. Verma, V., Fleisig, E., Tomlin, N., Klein, D.: Ghostbuster: Detecting Text Ghostwritten by Large Language Models. In: Duh, K., Gomez, H., Bethard, S. (eds.) Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). pp. 1702–1717. Association for Computational Linguistics, Mexico City, Mexico (Jun 2024). https://doi.org/10.18653/v1/2024.naacl-long.95, https://aclanthology.org/2024.naacl-long.95/
19. Wang, S., Li, Z., Qian, H., Yang, C., Wang, Z., Shang, M., Kumar, V., Tan, S., Ray, B., Bhatia, P., Nallapati, R., Ramanathan, M.K., Roth, D., Xiang, B.: ReCode: Robustness Evaluation of Code Generation Models. In: Rogers, A., Boyd-Graber, J., Okazaki, N. (eds.) Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 13818–13843. Association for Computational Linguistics, Toronto, Canada (Jul 2023). https://doi.org/10.18653/v1/2023.acl-long.773, https://aclanthology.org/2023.acl-long.773/
20. Wang, Y., Le, H., Gotmare, A., Bui, N., Li, J., Hoi, S.: CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In: Bouamor, H., Pino, J., Bali, K. (eds.) Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. pp. 1069–1088. Association for Computational

Linguistics, Singapore (Dec 2023). https://doi.org/10.18653/v1/2023.emnlp-main.68, https://aclanthology.org/2023.emnlp-main.68/

21. Wang, Y., Mansurov, J., Ivanov, P., Su, J., Shelmanov, A., Tsvigun, A., Whitehouse, C., Mohammed Afzal, O., Mahmoud, T., Sasaki, T., Arnold, T., Aji, A.F., Habash, N., Gurevych, I., Nakov, P.: M4: Multi-generator, Multi-domain, and Multi-lingual Black-Box Machine-Generated Text Detection. In: Graham, Y., Purver, M. (eds.) Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 1369–1407. Association for Computational Linguistics, St. Julian's, Malta (Mar 2024). https://doi.org/10.18653/v1/2024.eacl-long.83, https://aclanthology.org/2024.eacl-long.83/
22. Xu, X., Ni, C., Guo, X., Liu, S., Wang, X., Liu, K., Yang, X.: Distinguishing LLM-Generated from Human-Written Code by Contrastive Learning. ACM Transactions on Software Engineering and Methodology **34**(4), 1–31 (May 2025). https://doi.org/10.1145/3705300, https://dl.acm.org/doi/10.1145/3705300
23. Zhang, S., Zhao, H., Liu, X., Zheng, Q., Qi, Z., Gu, X., Dong, Y., Tang, J.: NaturalCodeBench: Examining Coding Performance Mismatch on HumanEval and Natural User Queries. In: Ku, L.W., Martins, A., Srikumar, V. (eds.) Findings of the Association for Computational Linguistics: ACL 2024. pp. 7907–7928. Association for Computational Linguistics, Bangkok, Thailand (Aug 2024). https://doi.org/10.18653/v1/2024.findings-acl.471, https://aclanthology.org/2024.findings-acl.471/
24. Zheng, Q., Xia, X., Zou, X., Dong, Y., Wang, S., Xue, Y., Shen, L., Wang, Z., Wang, A., Li, Y., Su, T., Yang, Z., Tang, J.: CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 5673–5684. ACM, Long Beach CA USA (Aug 2023). https://doi.org/10.1145/3580305.3599790, https://dl.acm.org/doi/10.1145/3580305.3599790